Ermeline Bresson n°12411807 *INFO2*
Bases de données avancées *Encadrant : M. Youcef*

TP5 : Transactions et contrôle de concurrence

Exercice 1 :

Ouverture de S1 et S2 et exécution de set autocommit off :

```
SQL> set autocommit off;
SQL>
```

1. Création de la table transaction depuis S1

```
SQL> create table transaction ( idTransaction VARCHAR2(44), valTransaction NUMBER(10));

Table cr??e.
```

2. Insertion, modification et suppression de lignes

Insertion de transactions :

```
SQL> insert into transaction values ('T1', 1);

1 ligne cr??e.

SQL> insert into transaction values ('T2', 2);

1 ligne cr??e.

SQL> insert into transaction values ('T3', 3);

1 ligne cr??e.
```

Modification de transaction :

```
SQL> update transaction set valTransaction = 33 where idTransaction = 'T1';

1 ligne mise ►jour.
```

Suppression d'une transaction :

```
SQL> delete from transaction where idTransaction = 'T2';

1 ligne supprim►e.
```

Affichage de la table avant Rollback :

```
IDTRANSACTION                                  VALTRANSACTION
---------------------------------------------- --------------
T1                                                         33
T3                                                          3
```

Exécution de Rollback :

```
SQL> rollback;

Annulation (rollback) effectu►e.
```

Affichage de la table :

```
SQL> select * from transaction;

aucune ligne s►lectionn►e

SQL>
```

Aucun ajout n'a été pris en compte.

3. Nouvelle insertion dans la table transaction

```
SQL> insert into transaction values ('T4', 4);

1 ligne cr►►e.

SQL> insert into transaction values ('T5', 5);

1 ligne cr►►e.
```

Exécution de la commande quit :

```
SQL> quit;
D►connect► de Oracle Database 21c Enterprise Edition Release 21.0.0.0.0 - Produc
tion
e12411807@g211-15:~$
```

Affichage de la table depuis S1 :

```
SQL> select * from transaction;

IDTRANSACTION                                   VALTRANSACTION
----------------------------------------------- --------------
T4                                                           4
T5                                                           5
```

Les ajouts ont bien été pris en compte.

4. Insertion et fermeture brutale de session

Réinsertion depuis S1 :

```
SQL>
SQL> insert into transaction values ('T6', 6);

1 ligne cr►►.
```

Affichage de la table après sortie brutale de la session 1 :

```
SQL> select * from transaction;

IDTRANSACTION                                   VALTRANSACTION
----------------------------------------------- --------------
T4                                                           4
T5                                                           5
```

Les données n'ont pas été sauvegardées.

5. Insertion à partir d'une nouvelle session

```
SQL> insert into transaction values ('T7', 7);

1 ligne cr►►.

SQL>
```

Modification de la table :

```
SQL> alter table transaction add val2transaction NUMBER(10);

Table modifi►.
```

Exécution de rollback :

```
SQL> rollback;

Annulation (rollback) effectu►.
```

Affichage de la table depuis S1 :

```
SQL> select * from transaction;

IDTRANSACTION                                  VALTRANSACTION VAL2TRANSACTION
---------------------------------------------- -------------- ---------------
T4                                                          4
T5                                                          5
T7                                                          7
```

La colonne val2transaction existe bien et la ligne T7 est conservée car avec oracle les commandes de type CREATE, ALTER, DROP font un commit automatique. Il aurait fallu remettre set autocommit off pour qu'elles ne soient pas prises en compte.

Conclusion :

Une session correspond à une connexion entre un utilisateur et la base de données. Chaque session est indépendante et possède son propre contexte d'exécution. Une transaction est un ensemble d'opérations (INSERT, UPDATE, DELETE) exécutées. Pour une transaction, toutes les opérations sont soit entièrement validées, soit entièrement annulées.

La validation d'une transaction se fait à l'aide de la commande COMMIT; qui rend les modifications permanentes. L'annulation d'une transaction se fait avec la commande ROLLBACK; qui supprime toutes les modifications non validées.

Certaines commandes, notamment les commandes CREATE, DROP, ALTER... entraînent automatiquement un commit, rendant les modifications irréversibles même en cas de rollback effectué par la suite.

Exercice 2 :

Création table de la table vol et de la table client :

```
SQL> create table vol (idVol VARCHAR2(44), capaciteVol NUMBER(10), nbPlacesReserveesVol NUMBER(10));

SQL> create table client ( idClient VARCHAR2(44), prenomClient VARCHAR2(11), nbPlacesReserveesClient NUMBER(10));

Table cr??e.
```

Ajout de données dans chacune des tables :

```
SQL> insert into vol values ('V1', 100, 0);

1 ligne cr??e.

SQL> insert into client values ('C1', 'Alice', 0);

1 ligne cr??e.

SQL> insert into client values ('C2', 'Bob', 0);

1 ligne cr??e.

SQL> commit;

Validation effectu?e.
```

Vérification de l'ajout par affichage de la table vol et client :

```
SQL> select * from vol;

IDVOL                                          CAPACITEVOL NBPLACESRESERVEESVOL
---------------------------------------------- ----------- --------------------
V1                                                     100                    0

SQL> select * from client;

IDCLIENT                                       PRENOMCLIEN NBPLACESRESERVEESCLIENT
---------------------------------------------- ----------- -----------------------
C1                                             Alice                             0
C2                                             Bob                               0
```

Modifications des tables vol et client depuis S1 sans commit :

```
SQL> update vol set nbPlacesReserveesVol = nbPlacesReserveesVol + 1 where idVol = 'V1';

1 ligne mise ?jour.

SQL> update client set nbPlacesReserveesClient = nbPlacesReserveesClient + 1 where idClient = 'C1';

1 ligne mise ?jour.
```

Vérification par affichage des deux tables dans S1 :

```
SQL> select * from vol;

IDVOL                                      CAPACITEVOL NBPLACESRESERVEESVOL
------------------------------------------ ----------- --------------------
V1                                                 100                    1

SQL> select * from client;

IDCLIENT                                   PRENOMCLIEN NBPLACESRESERVEESCLIENT
------------------------------------------ ----------- -----------------------
C1                                         Alice                             1
C2                                         Bob                               0
```

Les modifications ne s'affichent pas dans S2 :

```
SQL> select * from vol;

IDVOL                                      CAPACITEVOL NBPLACESRESERVEESVOL
------------------------------------------ ----------- --------------------
V1                                                 100                    0

SQL> select * from client;

IDCLIENT                                   PRENOMCLIEN NBPLACESRESERVEESCLIENT
------------------------------------------ ----------- -----------------------
C1                                         Alice                             0
C2                                         Bob                               0
```

Après avoir ajouté "commit;" dans S1, l'affichage est bien modifié et le nombre de places réservées est bien passé à 1 dans la table Vol et également pour la cliente Alice dans la table client.

ISOLATION DES TRANSACTIONS :

Modification des deux tables dans S1 :

```
SQL> update vol set nbPlacesReserveesVol = nbPlacesReserveesVol + 1 where idVol = 'V1';

1 ligne mise ►jour.

SQL> update client set nbPlacesReserveesClient = nbPlacesReserveesClient + 1 where idClient = 'C1';

1 ligne mise ►jour.
```

Affichage des deux tables depuis S1 :

```
SQL> select * from vol;

IDVOL                                      CAPACITEVOL NBPLACESRESERVEESVOL
------------------------------------------ ----------- --------------------
V1                                                 100                    2

SQL> select * from client;

IDCLIENT                                   PRENOMCLIEN NBPLACESRESERVEESCLIENT
------------------------------------------ ----------- -----------------------
C1                                         Alice                             2
C2                                         Bob                               0

SQL>
```

Les modifications sont bien visibles depuis S1, même sans commit.

```
SQL> select * from vol;

IDVOL                                      CAPACITEVOL NBPLACESRESERVEESVOL
------------------------------------------ ----------- --------------------
V1                                                 100                    1

SQL> select * from client;

IDCLIENT                                   PRENOMCLIEN NBPLACESRESERVEESCLIENT
------------------------------------------ ----------- -----------------------
C1                                         Alice                             1
C2                                         Bob                               0
```

Les modifications ne sont cependant pas visibles depuis S2.

Annulation depuis S1 :

```
SQL> rollback;

Annulation (rollback) effectu.
```

Les modifications ont bien été retirées :

```
SQL> select * from vol;

IDVOL                                          CAPACITEVOL NBPLACESRESERVEESVOL
---------------------------------------------- ----------- --------------------
V1                                                     100                    1

SQL> select * from client;

IDCLIENT                                       PRENOMCLIEN NBPLACESRESERVEESCLIENT
---------------------------------------------- ----------- -----------------------
C1                                             Alice                             1
C2                                             Bob                               0
```

Dans S2 rien n'a changé, comme attendu.

Nouvelle modification dans S1 :

```
SQL> UPDATE vol
  2  SET nbPlacesReserveesVol = nbPlacesReserveesVol + 1
  3  WHERE idVol = 'V1';

1 ligne mise ►jour.

SQL> UPDATE client
  2  SET nbPlacesReserveesClient = nbPlacesReserveesClient + 1
  3  WHERE idClient = 'C1';

1 ligne mise ►jour.
```

Ajout d'un commit pour valider les modifications effectuées :

```
SQL> commit;

Validation effectu►e.
```

Affichage de la table depuis S1 :

```
SQL> select * from vol;

IDVOL                                          CAPACITEVOL NBPLACESRESERVEESVOL
---------------------------------------------- ----------- --------------------
V1                                                     100                    2

SQL> select * from client;

IDCLIENT                                       PRENOMCLIEN NBPLACESRESERVEESCLIENT
---------------------------------------------- ----------- -----------------------
C1                                             Alice                             2
C2                                             Bob                               0
```

Les modifications ont bien été effectuées dans S1.

Elles sont également visibles depuis S2 :

```
SQL> select * from vol;

IDVOL                                          CAPACITEVOL NBPLACESRESERVEESVOL
---------------------------------------------- ----------- --------------------
V1                                                     100                    2

SQL> select * from client;

IDCLIENT                                       PRENOMCLIEN NBPLACESRESERVEESCLIENT
---------------------------------------------- ----------- -----------------------
C1                                             Alice                             2
C2                                             Bob                               0
```

Ainsi, une transaction voit immédiatement ses propres modifications, même si elles ne sont pas encore validées. Les autres transactions ne voient pas ces modifications tant qu'elles ne sont pas validées par un commit. L'utilisation de `ROLLBACK` permet d'annuler les modifications non validées, ramenant la table à son état initial. Une fois les modifications validées avec `COMMIT`, elles deviennent définitives et visibles pour toutes les autres sessions.

ISOLATION INCOMPLETE = INCOHERENCE POSSIBLE

Modification de la table :

```
SQL> UPDATE vol SET nbPlacesReserveesVol = 0 WHERE idVol = 'V1';

1 ligne mise ►jour.

SQL> UPDATE client SET nbPlacesReserveesClient = 0;

2 lignes mises ►jour.

SQL> COMMIT;

Validation effectu►e.
```

Transaction T1(S1) et T2(S2) :

```
SQL> SELECT nbPlacesReserveesVol FROM vol WHERE idVol = 'V1';

NBPLACESRESERVEESVOL
--------------------
                   0
```

```
SQL> SELECT nbPlacesReserveesClient FROM client WHERE idClient = 'C1';

NBPLACESRESERVEESCLIENT
-----------------------
                      0
```

Affichage depuis S2 :

```
SQL> SELECT nbPlacesReserveesVol FROM vol WHERE idVol = 'V1';

NBPLACESRESERVEESVOL
--------------------
                   0

SQL> SELECT nbPlacesReserveesClient FROM client WHERE idClient = 'C2';

NBPLACESRESERVEESCLIENT
-----------------------
                      0
```

Les deux transactions ne voient aucun billet réservé pour le vol et leurs clients.

Mise à jour de T1 et commit :

```
SQL> UPDATE vol
  2  SET nbPlacesReserveesVol = nbPlacesReserveesVol + 2
  3  WHERE idVol = 'V1';

1 ligne mise ►jour.

SQL> UPDATE client
  2  SET nbPlacesReserveesClient = nbPlacesReserveesClient + 2
  3  WHERE idClient = 'C1';

1 ligne mise ►jour.

SQL> COMMIT;

Validation effectu►e.
```

Modification depuis S2 :

```
SQL> UPDATE vol
  2  SET nbPlacesReserveesVol = nbPlacesReserveesVol + 3
  3  WHERE idVol = 'V1';

1 ligne mise ►jour.

SQL> UPDATE client
  2  SET nbPlacesReserveesClient = nbPlacesReserveesClient + 3
  3  WHERE idClient = 'C2';

1 ligne mise ►jour.

SQL> COMMIT;

Validation effectu►e.
```

Aucune incohérence n'a été observée car T2 a lu les données après que T1 ait été validée.
Le problème des mises à jour dépend de l'ordre exact des lectures et des commits.

ISOLATION COMPLÈTE = BLOCAGE ET REJET DES TRANSACTIONS POSSIBLES

Changement du niveau d'isolation en sérialisable depuis S1 et S2 :

```
SQL> alter session set isolation_level = serializable;

Session modifi►.
```

Réinitialisation des tables :

```
SQL> update vol set nbPlacesReserveesVol = 0 where idVol = 'V1';

1 ligne mise ►jour.

SQL> update client set nbPlacesReserveesClient = 0;

2 lignes mises ►jour.

SQL> commit;

Validation effectu►.
```

Lecture vol et client sur S1 :

```
SQL> SELECT nbPlacesReserveesVol FROM vol WHERE idVol='V1';

NBPLACESRESERVEESVOL
--------------------
                   0

SQL> SELECT nbPlacesReserveesClient FROM client WHERE idClient='C1';

NBPLACESRESERVEESCLIENT
-----------------------
                      0
```

Modification depuis S1 sans commit :

```
SQL> UPDATE vol SET nbPlacesReserveesVol = nbPlacesReserveesVol + 2 WHERE idVol='V1';

1 ligne mise ►jour.

SQL> UPDATE client SET nbPlacesReserveesClient = nbPlacesReserveesClient + 2 WHERE idClient='C1';

1 ligne mise ►jour.
```

Même chose depuis S2 :

```
SQL> SELECT nbPlacesReserveesVol FROM vol WHERE idVol='V1';

NBPLACESRESERVEESVOL
--------------------
                   0

SQL> SELECT nbPlacesReserveesClient FROM client WHERE idClient='C2';

NBPLACESRESERVEESCLIENT
-----------------------
                      0
```

Commit depuis S1 :

```
SQL> commit;

Validation effectu.
```

Commit depuis S2 :

```
SQL> commit;

Validation effectu.
```

Sous le paramètre par défaut d'Oracle, chaque transaction ne voit que les données validées au moment de sa lecture. Les mises à jour effectuées par d'autres transactions concurrentes peuvent ne pas être visibles immédiatement. Cela peut entraîner des mises à jour perdues si plusieurs transactions modifient la même donnée en parallèle. Ici, aucune incohérence n'a été observée car les lectures et commits se sont déroulées dans un ordre qui a préservé la cohérence. En effet, le total des billets réservés dans vol correspondait bien à la somme des réservations clients. Avec SERIALIZABLE chaque transaction voit un état de la base cohérent, comme si elle était exécutée en série. Ici, le deuxième commit a fonctionné car T2 lisait les données après que T1 ait commit, évitant tout conflit réel. Cela illustre que le rejet ou blocage dépend du temps exact des lectures et mises à jour. Ainsi, le niveau d'isolation détermine la visibilité des modifications entre transactions et la cohérence des données.